

12.2 Quick-Sort

The next sorting algorithm we discuss is called **quick-sort**. Like merge-sort, this algorithm is also based on the **divide-and-conquer** paradigm, but it uses this technique in a somewhat opposite manner, as all the hard work is done **before** the recursive calls.

High-Level Description of Quick-Sort

The quick-sort algorithm sorts a sequence S using a simple recursive approach. The main idea is to apply the divide-and-conquer technique, whereby we divide S into subsequences, recur to sort each subsequence, and then combine the sorted subsequences by a simple concatenation. In particular, the quick-sort algorithm consists of the following three steps (see Figure 12.8):

1. **Divide:** If S has at least two elements (nothing needs to be done if S has zero or one element), select a specific element x from S , which is called the **pivot**. As is common practice, choose the pivot x to be the last element in S . Remove all the elements from S and put them into three sequences:
 - L , storing the elements in S less than x
 - E , storing the elements in S equal to x
 - G , storing the elements in S greater than xOf course, if the elements of S are distinct, then E holds just one element—the pivot itself.
2. **Conquer:** Recursively sort sequences L and G .
3. **Combine:** Put back the elements into S in order by first inserting the elements of L , then those of E , and finally those of G .

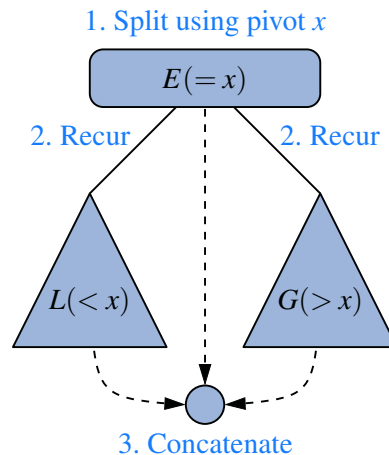


Figure 12.8: A visual schematic of the quick-sort algorithm.

Like merge-sort, the execution of quick-sort can be visualized by means of a binary recursion tree, called the *quick-sort tree*. Figure 12.9 summarizes an execution of the quick-sort algorithm by showing the input and output sequences processed at each node of the quick-sort tree. The step-by-step evolution of the quick-sort tree is shown in Figures 12.10, 12.11, and 12.12.

Unlike merge-sort, however, the height of the quick-sort tree associated with an execution of quick-sort is linear in the worst case. This happens, for example, if the sequence consists of n distinct elements and is already sorted. Indeed, in this case, the standard choice of the last element as pivot yields a subsequence L of size $n - 1$, while subsequence E has size 1 and subsequence G has size 0. At each call of quick-sort on subsequence L , the size decreases by 1. Hence, the height of the quick-sort tree is $n - 1$.

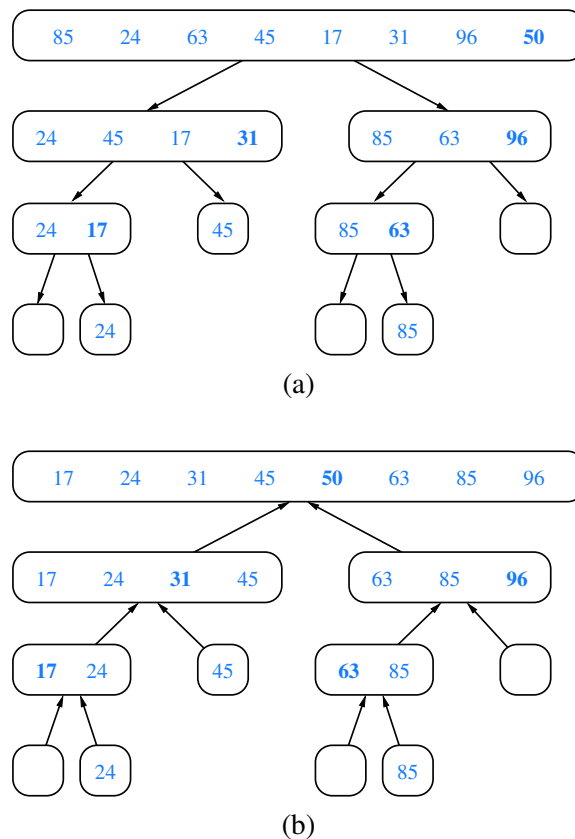


Figure 12.9: Quick-sort tree T for an execution of the quick-sort algorithm on a sequence with 8 elements: (a) input sequences processed at each node of T ; (b) output sequences generated at each node of T . The pivot used at each level of the recursion is shown in bold.

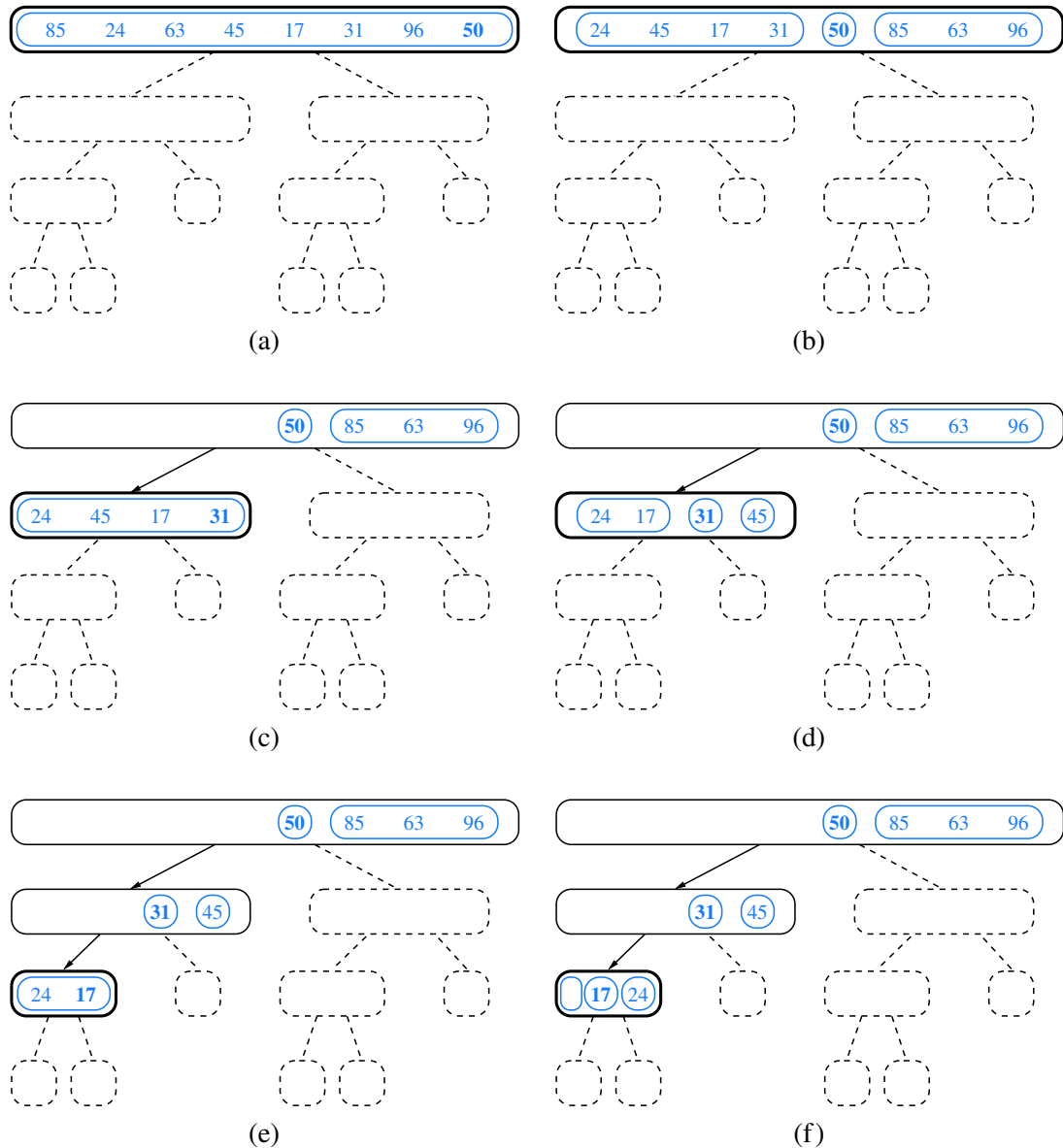


Figure 12.10: Visualization of quick-sort. Each node of the tree represents a recursive call. The nodes drawn with dashed lines represent calls that have not been made yet. The node drawn with thick lines represents the running call. The empty nodes drawn with thin lines represent terminated calls. The remaining nodes represent suspended calls (that is, active calls that are waiting for a child call to return). Note the divide steps performed in (b), (d), and (f). (Continues in Figure 12.11.)

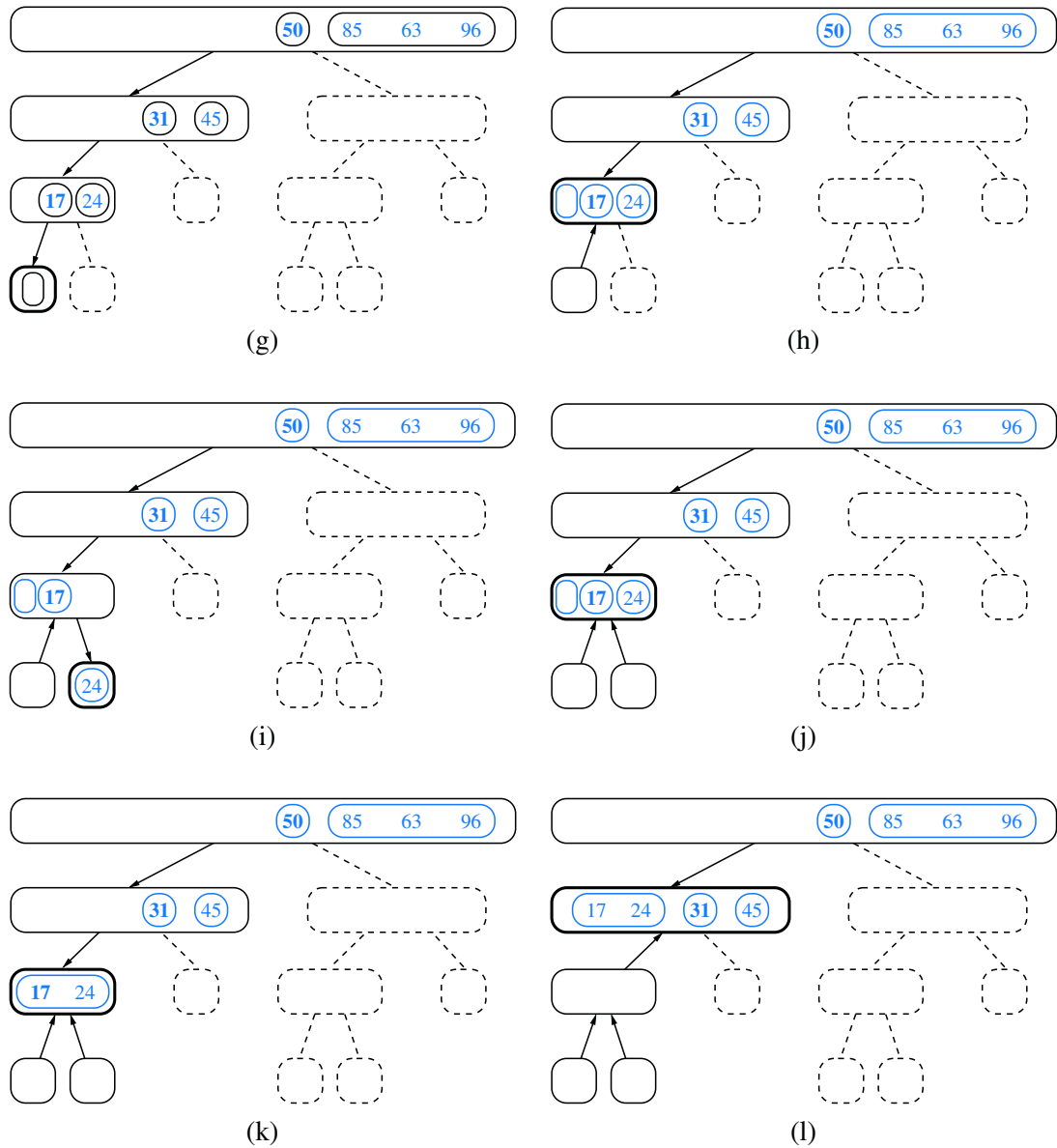


Figure 12.11: Visualization of an execution of quick-sort. Note the concatenation step performed in (k). (Continues in Figure 12.12.)

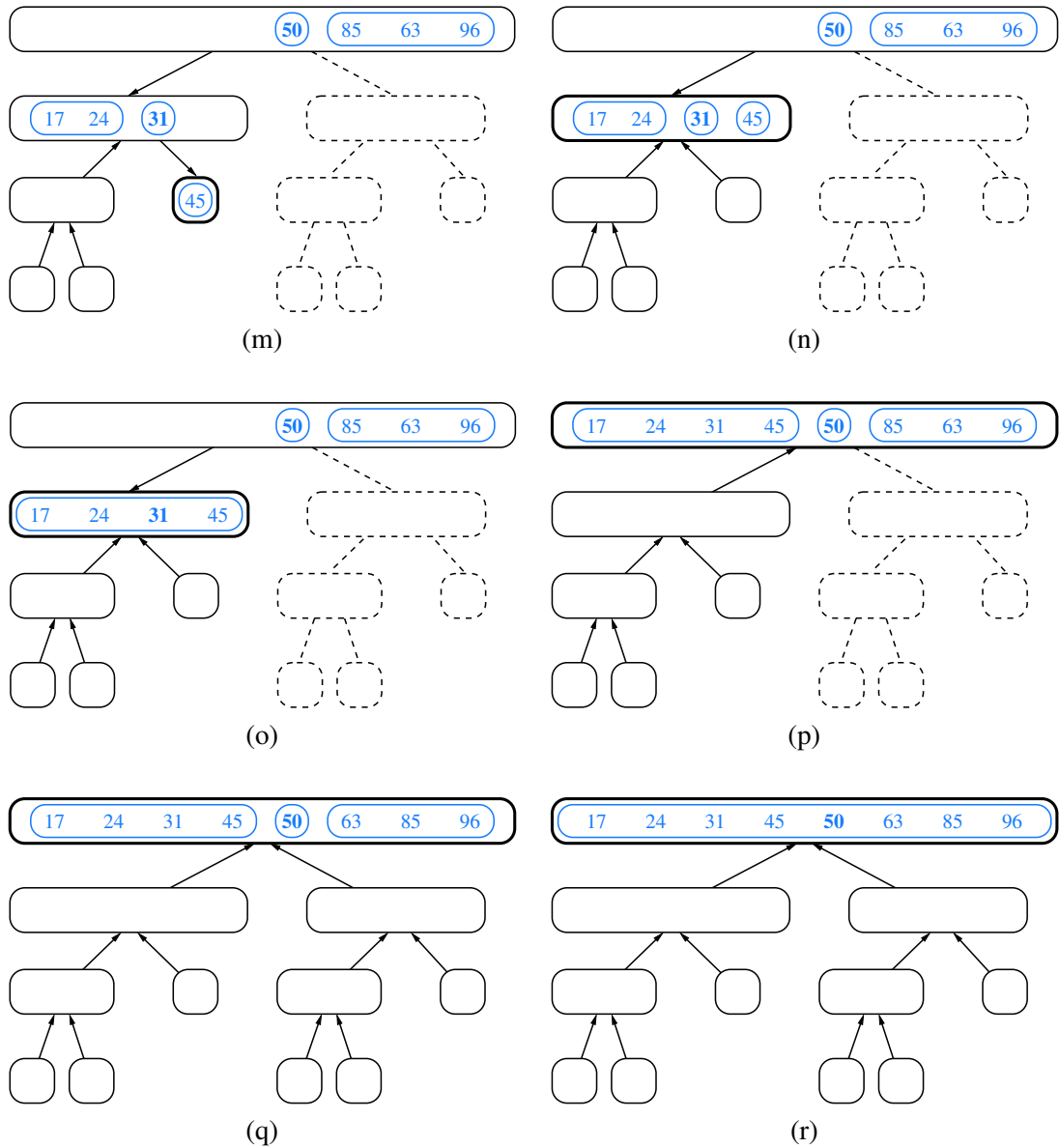


Figure 12.12: Visualization of an execution of quick-sort. Several calls between (p) and (q) have been omitted. Note the concatenation steps performed in (o) and (r). (Continued from Figure 12.11.)

Performing Quick-Sort on General Sequences

In Code Fragment 12.5, we give an implementation of the quick-sort algorithm that works on any sequence type that operates as a queue. This particular version relies on the `LinkedList` class from Section 6.2.3; we provide a more streamlined implementation of quick-sort using an array-based sequence in Section 12.2.2.

Our implementation chooses the first item of the queue as the pivot (since it is easily accessible), and then it divides sequence S into queues L , E , and G of elements that are respectively less than, equal to, and greater than the pivot. We then recur on the L and G lists, and transfer elements from the sorted lists L , E , and G back to S . All of the queue operations run in $O(1)$ worst-case time when implemented with a linked list.

```

1  /** Quick-sort contents of a queue. */
2  public static <K> void quickSort(Queue<K> S, Comparator<K> comp) {
3      int n = S.size();
4      if (n < 2) return;                // queue is trivially sorted
5      // divide
6      K pivot = S.first();              // using first as arbitrary pivot
7      Queue<K> L = new LinkedList<>();
8      Queue<K> E = new LinkedList<>();
9      Queue<K> G = new LinkedList<>();
10     while (!S.isEmpty()) {            // divide original into L, E, and G
11         K element = S.dequeue();
12         int c = comp.compare(element, pivot);
13         if (c < 0)                     // element is less than pivot
14             L.enqueue(element);
15         else if (c == 0)               // element is equal to pivot
16             E.enqueue(element);
17         else                           // element is greater than pivot
18             G.enqueue(element);
19     }
20     // conquer
21     quickSort(L, comp);                // sort elements less than pivot
22     quickSort(G, comp);                // sort elements greater than pivot
23     // concatenate results
24     while (!L.isEmpty())
25         S.enqueue(L.dequeue());
26     while (!E.isEmpty())
27         S.enqueue(E.dequeue());
28     while (!G.isEmpty())
29         S.enqueue(G.dequeue());
30 }
```

Code Fragment 12.5: Quick-sort for a sequence S implemented as a queue.

Running Time of Quick-Sort

We can analyze the running time of quick-sort with the same technique used for merge-sort in Section 12.1.3. Namely, we can identify the time spent at each node of the quick-sort tree T and sum up the running times for all the nodes.

Examining Code Fragment 12.5, we see that the divide step and the final concatenation of quick-sort can be implemented in linear time. Thus, the time spent at a node v of T is proportional to the *input size* $s(v)$ of v , defined as the size of the sequence handled by the call of quick-sort associated with node v . Since subsequence E has at least one element (the pivot), the sum of the input sizes of the children of v is at most $s(v) - 1$.

Let s_i denote the sum of the input sizes of the nodes at depth i for a particular quick-sort tree T . Clearly, $s_0 = n$, since the root r of T is associated with the entire sequence. Also, $s_1 \leq n - 1$, since the pivot is not propagated to the children of r . More generally, it must be that $s_i < s_{i-1}$ since the elements of the subsequences at depth i all come from distinct subsequences at depth $i - 1$, and at least one element from depth $i - 1$ does not propagate to depth i because it is in a set E (in fact, one element from *each node* at depth $i - 1$ does not propagate to depth i).

We can therefore bound the overall running time of an execution of quick-sort as $O(n \cdot h)$ where h is the overall height of the quick-sort tree T for that execution. Unfortunately, in the worst case, the height of a quick-sort tree is $n - 1$, as observed in Section 12.2. Thus, quick-sort runs in $O(n^2)$ worst-case time. Paradoxically, if we choose the pivot as the last element of the sequence, this worst-case behavior occurs for problem instances when sorting should be easy—if the sequence is already sorted.

Given its name, we would expect quick-sort to run quickly, and it often does in practice. The best case for quick-sort on a sequence of distinct elements occurs when subsequences L and G have roughly the same size. In that case, as we saw with merge-sort, the tree has height $O(\log n)$ and therefore quick-sort runs in $O(n \log n)$ time; we leave the justification of this fact as an exercise (R-12.12). More so, we can observe an $O(n \log n)$ running time even if the split between L and G is not as perfect. For example, if every divide step caused one subsequence to have one-fourth of those elements and the other to have three-fourths of the elements, the height of the tree would remain $O(\log n)$ and thus the overall performance $O(n \log n)$.

We will see in the next section that introducing randomization in the choice of a pivot will makes quick-sort essentially behave in this way on average, with an expected running time that is $O(n \log n)$.

12.2.1 Randomized Quick-Sort

One common method for analyzing quick-sort is to assume that the pivot will always divide the sequence in a reasonably balanced manner. However, we feel such an assumption would presuppose knowledge about the input distribution that is typically not available. For example, we would have to assume that we will rarely be given “almost” sorted sequences to sort, which are actually common in many applications. Fortunately, this assumption is not needed in order for us to match our intuition to quick-sort’s behavior.

In general, we desire some way of getting close to the best-case running time for quick-sort. The way to get close to the best-case running time, of course, is for the pivot to divide the input sequence S almost equally. If this outcome were to occur, then it would result in a running time that is asymptotically the same as the best-case running time. That is, having pivots close to the “middle” of the set of elements leads to an $O(n \log n)$ running time for quick-sort.

Picking Pivots at Random

Since the goal of the partition step of the quick-sort method is to divide the sequence S with sufficient balance, let us introduce randomization into the algorithm and pick as the pivot a **random element** of the input sequence. That is, instead of picking the pivot as the first or last element of S , we pick an element of S at random as the pivot, keeping the rest of the algorithm unchanged. This variation of quick-sort is called **randomized quick-sort**. The following proposition shows that the expected running time of randomized quick-sort on a sequence with n elements is $O(n \log n)$. This expectation is taken over all the possible random choices the algorithm makes, and is independent of any assumptions about the distribution of the possible input sequences the algorithm is likely to be given.

Proposition 12.3: *The expected running time of randomized quick-sort on a sequence S of size n is $O(n \log n)$.*

Justification: Let S be a sequence with n elements and let T be the binary tree associated with an execution of randomized quick-sort on S . First, we observe that the running time of the algorithm is proportional to the number of comparisons performed. We consider the recursive call associated with a node of T and observe that during the call, all comparisons are between the pivot element and another element of the input of the call. Thus, we can evaluate the total number of comparisons performed by the algorithm as $\sum_{s \in S} C(x)$, where $C(x)$ is the number of comparisons involving x as a nonpivot element. Next, we will show that for every element $x \in S$, the expected value of $C(x)$ is $O(\log n)$. Since the expected value of a sum is the sum of the expected values of its terms, an $O(\log n)$ bound on the expected value of $C(x)$ implies that randomized quick-sort runs in expected $O(n \log n)$ time.

To show that the expected value of $C(x)$ is $O(n \log n)$ for any x , we fix an arbitrary element x and consider the path of nodes in the tree T associated with recursive calls for which x is part of the input sequence. (See Figure 12.13.) By definition, $C(x)$ is equal to that path length, as x will take part in one nonpivot comparison per level of the tree until it is chosen as the pivot or is the only element that remains.

Let n_d denote the input size for the node of that path at depth d of tree T , for $0 \leq d \leq C(x)$. Since all elements are in the initial recursive call, $n_0 = n$. We know that the input size for any recursive call is at least one less than the size of its parent, and thus that $n_{d+1} \leq n_d - 1$ for any $d < C(x)$. In the worst case, this implies that $C(x) \leq n - 1$, as the recursive process stops if $n_d = 1$ or if x is chosen as the pivot.

We can show the stronger claim that the expected value of $C(x)$ is $O(\log n)$ based on the random selection of a pivot at each level. The choice of pivot at depth d of this path is considered “good” if $n_{d+1} \leq 3n_d/4$. The choice of a pivot will be good with probability at least $1/2$, as there are at least $n_d/2$ elements in the input that, if chosen as pivot, will result in at least $n_d/4$ elements being placed in each subproblem, thereby leaving x in a group with at most $3n_d/4$ elements.

We conclude by noting that there can be at most $\log_{4/3} n$ such good pivot choices before x is isolated. Since a choice is good with probability at least $1/2$, the expected number of recursive calls before achieving $\log_{4/3} n$ good choices is at most $2 \log_{4/3} n$, which implies that $C(x)$ is $O(\log n)$. ■

With a more rigorous analysis, we can show that the running time of randomized quick-sort is $O(n \log n)$ with *high probability*. (See Exercise C-12.55.)

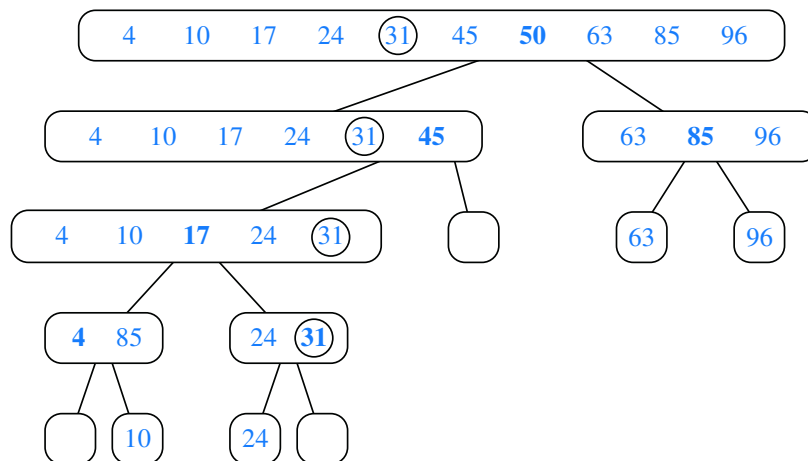


Figure 12.13: An illustration of the analysis of Proposition 12.3 for an execution of randomized quick-sort. We focus on element $x = 31$, which has value $C(x) = 3$, as it is the nonpivot element in a comparison with 50, 45, and 17. By our notation, $n_0 = 10$, $n_1 = 6$, $n_2 = 5$, and $n_3 = 2$, and the pivot choices of 50 and 17 are good.

12.2.2 Additional Optimizations for Quick-Sort

An algorithm is *in-place* if it uses only a small amount of memory in addition to that needed for the original input. Our implementation of heap-sort, from Section 9.4.2, is an example of such an in-place sorting algorithm. Our implementation of quick-sort from Code Fragment 12.5 does not qualify as in-place because we use additional containers L , E , and G when dividing a sequence S within each recursive call. Quick-sort of an array-based sequence can be adapted to be in-place, and such an optimization is used in most deployed implementations.

Performing the quick-sort algorithm in-place requires a bit of ingenuity, however, for we must use the input sequence itself to store the subsequences for all the recursive calls. We show algorithm `quickSortInPlace`, which performs in-place quick-sort of an array, in Code Fragment 12.6. In-place quick-sort modifies the input sequence using element swapping and does not explicitly create subsequences. Instead, a subsequence of the input sequence is implicitly represented by a range of positions specified by a leftmost index a and a rightmost index b . The divide

```

1  /** Sort the subarray S[a..b] inclusive. */
2  private static <K> void quickSortInPlace(K[] S, Comparator<K> comp,
3                                     int a, int b) {
4      if (a >= b) return;          // subarray is trivially sorted
5      int left = a;
6      int right = b - 1;
7      K pivot = S[b];
8      K temp;                      // temp object used for swapping
9      while (left <= right) {
10         // scan until reaching value equal or larger than pivot (or right marker)
11         while (left <= right && comp.compare(S[left], pivot) < 0) left++;
12         // scan until reaching value equal or smaller than pivot (or left marker)
13         while (left <= right && comp.compare(S[right], pivot) > 0) right--;
14         if (left <= right) {      // indices did not strictly cross
15             // so swap values and shrink range
16             temp = S[left]; S[left] = S[right]; S[right] = temp;
17             left++; right--;
18         }
19     }
20     // put pivot into its final place (currently marked by left index)
21     temp = S[left]; S[left] = S[b]; S[b] = temp;
22     // make recursive calls
23     quickSortInPlace(S, comp, a, left - 1);
24     quickSortInPlace(S, comp, left + 1, b);
25 }
```

Code Fragment 12.6: In-place quick-sort for an array S . The entire array can be sorted as `quickSortInPlace(S, comp, 0, S.length-1)`.

step is performed by scanning the array simultaneously using local variables *left*, which advances forward, and *right*, which advances backward, swapping pairs of elements that are in reverse order, as shown in Figure 12.14. When these two indices pass each other, the division step is complete and the algorithm completes by recurring on these two sublists. There is no explicit “combine” step, because the concatenation of the two sublists is implicit to the in-place use of the original list.

It is worth noting that if a sequence has duplicate values, we are not explicitly creating three sublists *L*, *E*, and *G*, as in our original quick-sort description. We instead allow elements equal to the pivot (other than the pivot itself) to be dispersed across the two sublists. Exercise R-12.11 explores the subtlety of our implementation in the presence of duplicate keys, and Exercise C-12.34 describes an in-place algorithm that strictly partitions into three sublists *L*, *E*, and *G*.

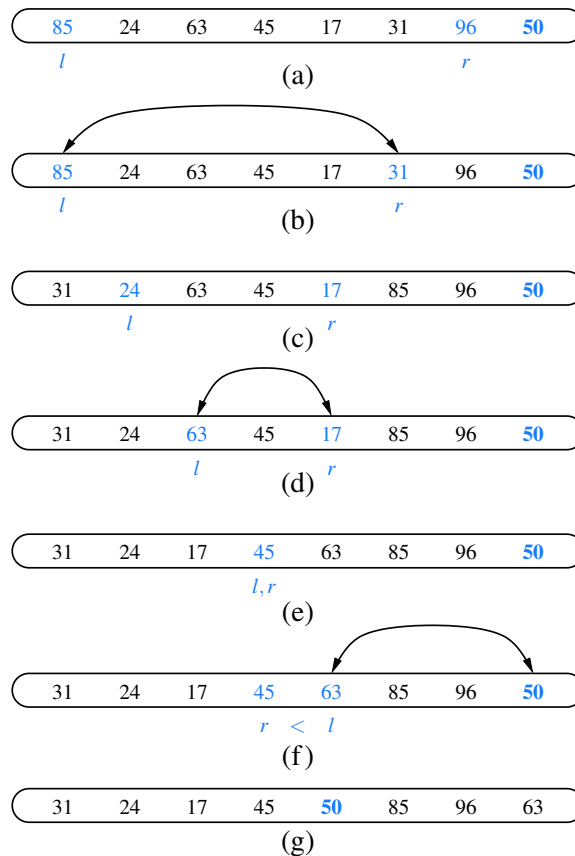


Figure 12.14: Divide step of in-place quick-sort, using index *l* as shorthand for identifier left, and index *r* as shorthand for identifier right. Index *l* scans the sequence from left to right, and index *r* scans the sequence from right to left. A swap is performed when *l* is at an element as large as the pivot and *r* is at an element as small as the pivot. A final swap with the pivot, in part (f), completes the divide step.

Although the implementation we describe in this section for dividing the sequence into two pieces is in-place, we note that the complete quick-sort algorithm needs space for a stack proportional to the depth of the recursion tree, which in this case can be as large as $n - 1$. Admittedly, the expected stack depth is $O(\log n)$, which is small compared to n . Nevertheless, a simple trick lets us guarantee the stack size is $O(\log n)$. The main idea is to design a nonrecursive version of in-place quick-sort using an explicit stack to iteratively process subproblems (each of which can be represented with a pair of indices marking subarray boundaries). Each iteration involves popping the top subproblem, splitting it in two (if it is big enough), and pushing the two new subproblems. The trick is that when pushing the new subproblems, we should first push the larger subproblem and then the smaller one. In this way, the sizes of the subproblems will at least double as we go down the stack; hence, the stack can have depth at most $O(\log n)$. We leave the details of this implementation as an exercise (P-12.59).

Pivot Selection

Our implementation in this section blindly picks the last element as the pivot at each level of the quick-sort recursion. This leaves it susceptible to the $\Theta(n^2)$ -time worst case, most notably when the original sequence is already sorted, reverse sorted, or nearly sorted.

As described in Section 12.2.1, this can be improved upon by using a randomly chosen pivot for each partition step. In practice, another common technique for choosing a pivot is to use the median of three values, taken respectively from the front, middle, and tail of the array. This *median-of-three* heuristic will more often choose a good pivot and computing a median of three may require lower overhead than selecting a pivot with a random number generator. For larger data sets, the median of more than three potential pivots might be computed.

Hybrid Approaches

Although quick-sort has very good performance on large data sets, it has rather high overhead on relatively small data sets. For example, the process of quick-sorting a sequence of eight elements, as illustrated in Figures 12.10 through 12.12, involves considerable bookkeeping. In practice, a simple algorithm like insertion-sort (Section 7.6) will execute faster when sorting such a short sequence.

It is therefore common, in optimized sorting implementations, to use a hybrid approach, with a divide-and-conquer algorithm used until the size of a subsequence falls below some threshold (perhaps 50 elements); insertion-sort can be directly invoked upon portions with length below the threshold. We will further discuss such practical considerations in Section 12.4, when comparing the performance of various sorting algorithms.